

Ya2yOS

基于 Rust 的 Linux 兼容宏内核



RISC-V64

LoongArch64

ext4 / smoltcp / VirtIO

项目定位

目标

提供可运行真实 Linux 用户态负载的实验内核，而不是只服务教学样例。

架构

宏内核设计，核心子系统运行在内核态并共享内核地址空间。

基础

以 TatlinOS 为基础，使用 Rust 重构和扩展核心路径。

验证

面向 BusyBox、glibc/musl、libc-test、LTP、lmbench、iozone 等测试持续补齐。

总体能力

系统调用

200+ Linux syscall, 覆盖进程、内存、文件、网络、信号、同步和时间。

进程模型

fork/clone/clone3、execve、exit/wait、线程组、进程组和信号事件。

内存管理

SV39/LA64 页表、mmap/brk、COW、lazy allocation、共享映射。

文件系统

VFS + lwext4, 统一普通文件、设备、管道、socket 和事件对象。

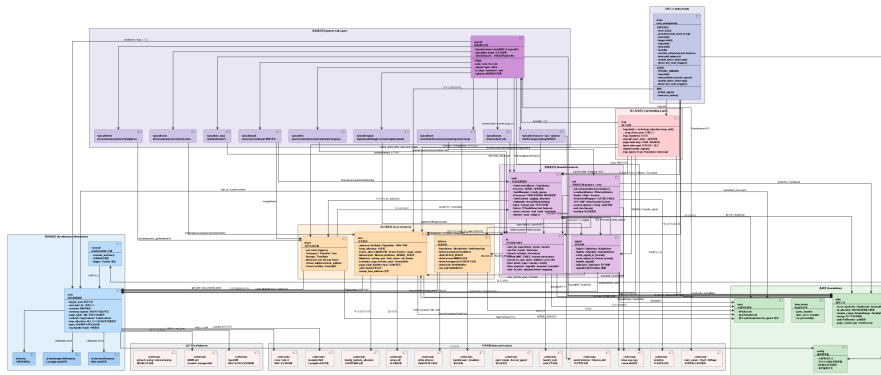
网络

smoltcp TCP/UDP、loopback、VirtIO-net、Unix domain socket。

设备

VirtIO blk/net, RISC-V64 MMIO 与 LoongArch64 PCI 双路径。

架构分层

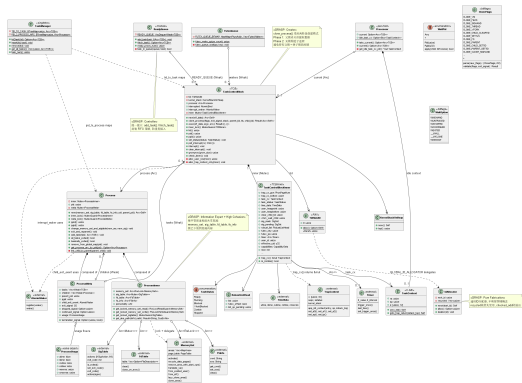


arch / trap / syscall

task / mm / fs / net

drivers / signal / timer

进程与调度



核心划分

Process 管资源，TaskControlBlock 管执行。同一进程的线程通过 Arc<Process> 共享地址空间、fd 表和信号表。

调度路径

每 Hart 一个 Processor，全局 ready queue 保存可运行任务，__switch 隔离架构相关上下文切换。

Linux 语义

支持 wait4/waitid、SIGCHLD、stopped/continued/killed/core dumped 等等待事件。

内存管理

地址空间

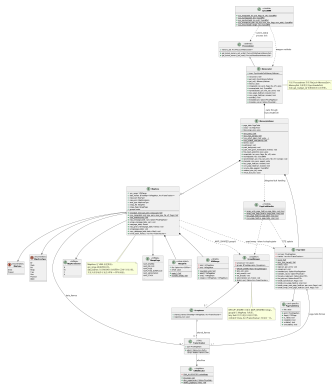
用户空间包含 ELF、brk、动态链接器、mmap、用户栈和 TrapContext；内核高地址 direct mapping 物理内存。

COW

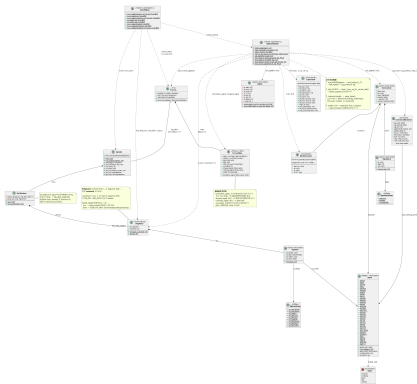
fork 共享可写页并清写权限；写缺页时按引用计数选择恢复权限或深拷贝。

mmap

VMA 拆分、权限更新、匿名页与文件页 lazy fault、MAP_SHARED 脏页写回。



信号机制



投递

kill/tkill/tgkill 支持线程组或单线程目标；阻塞任务收到信号会被唤醒。

处理

返回用户态前检查 pending & !mask，默认动作可忽略、终止、停止或继续。

信号帧

自定义 handler 在用户栈构建 sigframe，
rt_sigreturn 恢复寄存器上下文和旧 mask。

文件系统

VFS

SuperBlock、Inode、File 三层抽象，将 ext4、设备、管道、socket、epoll/eventfd 统一为 fd。

fd 表

FdTable 管理 close-on-exec、dup、资源限制；FSInfo 管 cwd、exe、umask 和 fd 路径。

ext4

通过 lwext4_rust 接入真实 ext4，支持目录、软硬链接、truncate、stat、getdents64。

兼容对象

/dev/null、/dev/zero、/proc/<pid>、loop 设备、mqueue、inotify 和挂载上下文 fd。

网络与设备

网络分层

socket syscall
TCP / UDP / Unix socket
smoltcp Interface + SocketSet
Router + Loopback + Ethernet
VirtIO-net

设备抽象

BlockDriver 与 NetDriverOps 隔离上层文件系统、网络栈和具体 VirtIO 实现。

双架构

RISC-V64 使用 MMIO VirtIO；LoongArch64 通过 PCI 枚举创建 VirtIO transport。

当前策略

块设备同步 IO，网络由 poll_interfaces() 推进，后续可转向中断和 waker 驱动。

工程特色

兼容性优先

对核心 syscall 提供真实语义，对探测型接口提供明确兼容 stub，避免用户态过早失败。

Rust 生命周期

Arc、Weak、锁、trait 和 enum 分发降低复杂对象生命周期错误。

真实后端

ext4、VirtIO、smoltcp 让内核直接面对真实用户态程序和文件系统镜像。

文档化修复

problem 目录记录 LTP、libc-test 和 BusyBox 问题的现象、根因、修复与验证。

已知边界

方向	当前限制
页缓存	read/write 与 mmap 尚未通过统一 page cache 贯通。
挂载	挂载表偏兼容，尚未形成完整多 superblock 挂载树。
网络	依赖周期性轮询，VirtIO-net 中断与 socket waker 仍需完善。
syscall	部分低频接口仍为兼容 stub，需要逐步补真实语义。
调度	多核负载均衡、公平调度和抢占仍可继续增强。

后续方向

短期

统一页缓存、网络中断唤醒、真实 loop 设备、stub 分类。

中期

tmpfs/procfs/devtmpfs、完整挂载树、调度多核、AIO/io_uring。

长期

namespace/cgroup/capabilities、更多 VirtIO 设备、IPv6/netlink、自动化测试矩阵。

从“接口覆盖”走向“语义做深”

总结

Ya2yOS 已经形成从启动、进程、内存、文件、网络、信号到设备驱动的完整主线能力，并通过 RISC-V64 / LoongArch64 双架构验证了核心抽象。

后续重点是减少 stub、完善缓存和挂载语义、提升网络与 IO 事件驱动能力，并用持续测试保持双架构行为一致。